

Métodos directos para sistemas lineales

Clases 1 y 2 · Secciones 2.1 a 2.7

Análisis Numérico (LMA) · Métodos Numéricos II (LCD)

18 de agosto de 2026

Facultad de Ingeniería Química · UNL

Plan de la clase

1. El problema $Ax = b$: métodos directos vs. iterativos.
2. **2.1** Breve repaso de Álgebra Lineal.
3. **2.2** Matrices triangulares inferiores (sustitución hacia adelante).
4. **2.3** Matrices triangulares superiores (sustitución hacia atrás).
5. **2.4** Matrices llenas: método de Gauss.
6. **2.5** Factorización LU .
7. **2.6** Problemas con LU : pivoteo.
8. **2.7** Matrices sdp: factorización de Cholesky.

Hilo conductor: **costo computacional** (# de operaciones) vs. **estabilidad**.

El problema

Dada $A \in \mathbb{R}^{n \times n}$ no singular y $b \in \mathbb{R}^n$, hallar $x \in \mathbb{R}^n$ tal que

$$Ax = b.$$

El problema

Dada $A \in \mathbb{R}^{n \times n}$ **no singular** y $b \in \mathbb{R}^n$, hallar $x \in \mathbb{R}^n$ tal que

$$Ax = b.$$

¿Dónde aparece? splines cúbicos (matriz tridiagonal), optimización, discretización de ecuaciones diferenciales. . .

A menudo con matrices **grandes** y **ralas** (*sparse*): pocas entradas no nulas.

El problema

Dada $A \in \mathbb{R}^{n \times n}$ **no singular** y $b \in \mathbb{R}^n$, hallar $x \in \mathbb{R}^n$ tal que

$$Ax = b.$$

¿**Dónde aparece?** splines cúbicos (matriz tridiagonal), optimización, discretización de ecuaciones diferenciales. . .

A menudo con matrices **grandes** y **ralas** (*sparse*): pocas entradas no nulas.

Dos familias de métodos

Directos llegan a la solución en un número **finito** de pasos (sin errores de redondeo). Ej.: eliminación de Gauss. Para sistemas no muy grandes.

Iterativos generan $\{x_k\}_{k \in \mathbb{N}}$ con $x_k \rightarrow x$. Ej.: Jacobi, Gauss–Seidel, SOR, gradiente conjugado. Para sistemas muy grandes.

2.1 Breve repaso de Álgebra Lineal

Convención: los vectores son **columna**. Si $x, y \in \mathbb{R}^n$:

- Producto escalar: $x \cdot y = x^T y \in \mathbb{R}$.
- Producto tensorial: $x \otimes y = x y^T \in \mathbb{R}^{n \times n}$, de **rango uno**,

$$\text{fila}_i(xy^T) = x_i y^T, \quad \text{col}_j(xy^T) = y_j x.$$

Convención: los vectores son **columna**. Si $x, y \in \mathbb{R}^n$:

- Producto escalar: $x \cdot y = x^T y \in \mathbb{R}$.
- Producto tensorial: $x \otimes y = x y^T \in \mathbb{R}^{n \times n}$, de **rango uno**,

$$\text{fila}_i(xy^T) = x_i y^T, \quad \text{col}_j(xy^T) = y_j x.$$

Para $A \in \mathbb{R}^{n \times m}$, $B \in \mathbb{R}^{m \times p}$:

$$(AB)_{ij} = \text{fila}_i(A) \text{col}_j(B), \quad \text{fila}_i(AB) = \text{fila}_i(A) B, \quad \text{col}_j(AB) = A \text{col}_j(B).$$

Ax como combinación de columnas

Si $A \in \mathbb{R}^{n \times m}$ y $x \in \mathbb{R}^m$:

$$Ax = x_1 \operatorname{col}_1(A) + x_2 \operatorname{col}_2(A) + \cdots + x_m \operatorname{col}_m(A) \in \operatorname{col}(A).$$

Es decir, Ax vive en el **espacio columna** de A .

Ax como combinación de columnas

Si $A \in \mathbb{R}^{n \times m}$ y $x \in \mathbb{R}^m$:

$$Ax = x_1 \operatorname{col}_1(A) + x_2 \operatorname{col}_2(A) + \cdots + x_m \operatorname{col}_m(A) \in \operatorname{col}(A).$$

Es decir, Ax vive en el **espacio columna** de A .

Análogamente, $x^T A$ es combinación lineal de las **filas** de A .

Con e_i el i -ésimo canónico:

$$Ae_i = \operatorname{col}_i(A), \quad e_i^T A = \operatorname{fila}_i(A).$$

No singularidad: afirmaciones equivalentes

Para $A \in \mathbb{R}^{n \times n}$, son **equivalentes**:

- (1) Para todo $b \in \mathbb{R}^n$ existe x con $Ax = b$.
- (2) Si $Ax = b$ tiene solución, es única.
- (3) $Ax = 0 \implies x = 0$.
- (4) Las columnas de A son linealmente independientes.
- (5) Las filas de A son linealmente independientes.
- (6) Existe A^{-1} con $AA^{-1} = A^{-1}A = I$.
- (7) $\det(A) \neq 0$.

Observación

Si $A \in \mathbb{R}^{30 \times 30}$ con $\det(A) = 1$, entonces

$$\det(0.1 A) = (0.1)^{30} \det(A) = 10^{-30}.$$

Observación

Si $A \in \mathbb{R}^{30 \times 30}$ con $\det(A) = 1$, entonces

$$\det(0.1 A) = (0.1)^{30} \det(A) = 10^{-30}.$$

Un determinante **pequeño no indica** que la matriz sea “casi singular”.

La información correcta la da el **número de condición** (lo veremos más adelante).

2.2 Matrices triangulares inferiores

Definición

$L \in \mathbb{R}^{n \times n}$ es **triangular inferior** si $\ell_{ij} = 0$ para $i < j$; es **unitaria** si además $\ell_{ii} = 1$.

$$L = \begin{bmatrix} \ell_{11} & 0 & \cdots & 0 \\ \ell_{21} & \ell_{22} & & \vdots \\ \vdots & & \ddots & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{nn} \end{bmatrix}$$

Definición

$L \in \mathbb{R}^{n \times n}$ es **triangular inferior** si $\ell_{ij} = 0$ para $i < j$; es **unitaria** si además $\ell_{ii} = 1$.

$$L = \begin{bmatrix} \ell_{11} & 0 & \cdots & 0 \\ \ell_{21} & \ell_{22} & & \vdots \\ \vdots & & \ddots & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{nn} \end{bmatrix}$$

$\det(L) = \ell_{11} \cdots \ell_{nn}$: solución **única** de $Lx = b$ si $\ell_{ii} \neq 0 \ \forall i$.

Sustitución hacia adelante

De $Lx = b$, respetando el orden:

$$x_1 = b_1/\ell_{11}$$

$$x_2 = (b_2 - \ell_{21}x_1)/\ell_{22}$$

$$x_3 = (b_3 - \ell_{31}x_1 - \ell_{32}x_2)/\ell_{33}$$

$$\vdots$$

$$x_n = \left(b_n - \sum_{j=1}^{n-1} \ell_{nj}x_j\right)/\ell_{nn}.$$

Forward substitution. Se implementa por **filas** o por **columnas**.

Código: versión directa (estilo C/Fortran)

```
function x = triinf(L,b)
% Resuelve  $Lx = b$ , con  $L$  triangular inferior
% (usa solo  $L_{ij}$  con  $i \geq j$ ).
n = length(b);
x = b;
for i = 1:n
    for j = 1:i-1
        x(i) = x(i) - L(i,j) * x(j);
    end
    x(i) = x(i) / L(i,i);
end
```

En MATLAB/OCTAVE conviene **vectorizar**: evitar los *loops* internos.

Por filas (loop interno $\rightarrow$ producto escalar):

```
for i = 1:n
    x(i) = x(i) - L(i,1:i-1) * x(1:i-1);
    x(i) = x(i) / L(i,i);
end
```

Por columnas (actualiza el resto apenas se conoce x_i):

```
x(1) = x(1) / L(1,1);
for i = 2:n
    x(i:n) = x(i:n) - x(i-1) * L(i:n,i-1);
    x(i) = x(i) / L(i,i);
end
```

Conteo de operaciones

Para `triinf_cols`:

Momento	mult.	sumas	div.
antes del for			1
iteración k	$n - k$	$n - k$	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$
total	$\frac{(n-1)n}{2}$	$\frac{(n-1)n}{2}$	n

$$(n - 1)n + n = n^2 \text{ operaciones.}$$

- Cualquiera de nuestras versiones es **más lenta** que L\b: las rutinas internas ya están compiladas; triinf es *interpretado*.
- Las versiones **vectorizadas** son más rápidas.
- En C/Fortran/Python hay rutinas optimizadas (BLAS):

saxpy $z \leftarrow \alpha x + y$;

dot $a \leftarrow x \cdot y$.

Para vectores largos se **paralelizan**.

2.3 Matrices triangulares superiores

Triangular superior y sustitución hacia atrás

$U \in \mathbb{R}^{n \times n}$ es **triangular superior** si $u_{ij} = 0$ para $i > j$.

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{bmatrix}$$

Resolver $Ux = b$ es análogo, pero empezando por x_n : **sustitución hacia atrás** (*backward substitution*).

$$x_n = \frac{b_n}{u_{nn}}, \quad x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^n u_{ij} x_j \right).$$

Ejercicio

Implementar trisup por filas y por columnas. Costo: también n^2 .

2.4 Matrices llenas: método de Gauss

Idea del método de Gauss

Resolver $Ax = b$ con $A \in \mathbb{R}^{n \times n}$ invertible.

Operaciones elementales sobre A y b hasta obtener un sistema triangular superior equivalente, que se resuelve hacia atrás.

Idea del método de Gauss

Resolver $Ax = b$ con $A \in \mathbb{R}^{n \times n}$ invertible.

Operaciones elementales sobre A y b hasta obtener un sistema **triangular superior** equivalente, que se resuelve hacia atrás.

Primer paso ($A^{(0)} \rightarrow A^{(1)}$): ceros debajo de a_{11} .

$$m_{k1} = \frac{a_{k1}^{(0)}}{a_{11}^{(0)}}, \quad \text{fila}_k(A^{(1)}) = \text{fila}_k(A^{(0)}) - m_{k1} \text{fila}_1(A^{(0)}), \quad k = 2, \dots, n,$$

y lo mismo sobre b : $b_k^{(1)} = b_k^{(0)} - m_{k1} b_1^{(0)}$.

Paso j genérico

Multiplicadores y actualización de las filas $j + 1, \dots, n$:

$$m_{kj} = \frac{a_{kj}^{(j-1)}}{a_{jj}^{(j-1)}}, \quad \text{fila}_k(A^{(j)}) = \text{fila}_k(A^{(j-1)}) - m_{kj} \text{fila}_j(A^{(j-1)}), \quad k = j + 1, \dots, n,$$

$$b_k^{(j)} = b_k^{(j-1)} - m_{kj} b_j^{(j-1)}.$$

Paso j genérico

Multiplicadores y actualización de las filas $j + 1, \dots, n$:

$$m_{kj} = \frac{a_{kj}^{(j-1)}}{a_{jj}^{(j-1)}}, \quad \text{fila}_k(A^{(j)}) = \text{fila}_k(A^{(j-1)}) - m_{kj} \text{fila}_j(A^{(j-1)}), \quad k = j + 1, \dots, n,$$

$$b_k^{(j)} = b_k^{(j-1)} - m_{kj} b_j^{(j-1)}.$$

Al terminar el paso $n - 1$:

$$A^{(n-1)}_x = b^{(n-1)}, \quad A^{(n-1)} = U \text{ triangular superior.}$$

El **pivote** del paso j es $a_{jj}^{(j-1)}$ (debe ser $\neq 0$).

Costo del método de Gauss

Triangulación sobre A:

$$\sum_{j=1}^{n-1} (1 + 2(n-j))(n-j) = \frac{2}{3}n^3 - \frac{1}{2}n^2 - \frac{1}{6}n \cong \frac{2}{3}n^3.$$

Sobre b: $\sum_{j=1}^{n-1} 2(n-j) = n^2 - n$. Sustitución hacia atrás: n^2 .

Total

$$\frac{2}{3}n^3 + \frac{3}{2}n^2 - \frac{7}{6}n \cong \boxed{\frac{2}{3}n^3} \text{ operaciones.}$$

El término dominante $\frac{2}{3}n^3$ marca la **complejidad cúbica**.

2.5 Factorización LU

El paso j se escribe como producto por una matriz:

$$A^{(j)} = G^j A^{(j-1)}, \quad G^j = I - g_j e_j^T, \quad g_j = (0, \dots, 0, m_{(j+1)j}, \dots, m_{nj})^T.$$

Las G^j se llaman **matrices de Gauss**. Propiedad clave:

$$(G^j)^{-1} = (I - g_j e_j^T)^{-1} = I + g_j e_j^T.$$

De Gauss a $A = LU$

Como $U = G^{n-1} \dots G^1 A$,

$$(G^1)^{-1} \dots (G^{n-1})^{-1} = I + \sum_{j=1}^{n-1} g_j e_j^T = \begin{bmatrix} 1 & 0 & \dots & 0 \\ m_{21} & 1 & & \vdots \\ \vdots & & \ddots & 0 \\ m_{n1} & m_{n2} & \dots & 1 \end{bmatrix} = L.$$

Conclusión

Si $a_{jj}^{(j-1)} \neq 0$ para $j = 1, \dots, n-1$, Gauss produce

$$A = LU, \quad L \text{ tri. inf. unitaria}, \quad U \text{ tri. sup.}, \quad \ell_{ij} = m_{ij}.$$

$$Ax = b \iff L \underbrace{Ux}_y = b : \quad \underbrace{Ly = b}_{n^2 - n \text{ op.}} \text{ (adelante), } \underbrace{Ux = y}_{n^2 \text{ op.}} \text{ (atrás).}$$

$$Ax = b \iff L \underbrace{Ux}_y = b : \quad \underbrace{Ly = b}_{n^2 - n \text{ op.}} \text{ (adelante), } \underbrace{Ux = y}_{n^2 \text{ op.}} \text{ (atrás).}$$

Factorizar cuesta $\frac{2}{3}n^3$; resolver los dos triangulares, $2n^2 - n$.

$$\text{Total} = \frac{2}{3}n^3 + \frac{3}{2}n^2 - \frac{7}{6}n \cong \frac{2}{3}n^3,$$

lo mismo que Gauss (no sorprende: es Gauss reescrito).

L y U se guardan sobre A : el multiplicador m_{ij} ocupa el lugar del cero que produce.

¿Por qué factorizar? Muchos b , misma A

Resolver k sistemas con la misma A :

Gauss cada vez

$$k \left(\frac{2}{3} n^3 + \dots \right) \cong k \frac{2}{3} n^3.$$

Factorizar una vez

$$\frac{2}{3} n^3 + k (2n^2 - n) \cong \frac{2}{3} n^3 + k n^2.$$

Para k grande, factorizar una sola vez y reusar L, U es **mucho** más barato.

Código: factorización LU (vectorizada)

```
function [L,U] = eleu(A)
% Descomposicion LU:  $L*U = A$ 
% L triangular inferior unitaria, U triangular superior.
n = size(A,1);
L = eye(n);
for j = 1:n-1
    L(j+1:n,j) = A(j+1:n,j) / A(j,j);
    A(j+1:n,j) = 0;
    A(j+1:n,j+1:n) = A(j+1:n,j+1:n) - L(j+1:n,j)*A(j,j+1:n);
end
U = A;
```

Existe también la versión con doble for (más clara, menos eficiente).

2.6 Problemas con LU : pivoteo

El algoritmo puede fallar

Si $a_{jj}^{(j-1)} = 0$, no se puede dividir. Y a veces **no existe** LU :

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \neq \begin{bmatrix} 1 & 0 \\ \ell & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{bmatrix} \quad \text{para ningún } \ell, u_{ij}.$$

($u_{11} = 0$ forzaría la primera fila nula.)

Además, dividir por pivotes **pequeños** amplifica los errores de redondeo.

Solución: *pivoteo*.

Teorema (existencia y unicidad de LU)

Teorema

Sea $A \in \mathbb{R}^{n \times n}$. Si $\det(A_{1:k, 1:k}) \neq 0$ para todo $k = 1, \dots, n - 1$ (menores principales líderes no nulos), entonces existe la descomposición $A = LU$ con L tri. inf. unitaria y U tri. sup. Si además A es invertible, la descomposición es *única*.

Teorema (existencia y unicidad de LU)

Teorema

Sea $A \in \mathbb{R}^{n \times n}$. Si $\det(A_{1:k,1:k}) \neq 0$ para todo $k = 1, \dots, n-1$ (menores principales líderes no nulos), entonces existe la descomposición $A = LU$ con L tri. inf. unitaria y U tri. sup. Si además A es invertible, la descomposición es *única*.

Idea: inducción sobre j , mostrando que $a_{jj}^{(j-1)} \neq 0$; se usa que $\det(A_{1:k+1,1:k+1}^{(k)}) = a_{11}^{(0)} \cdots a_{(k+1)(k+1)}^{(k)}$. La unicidad sale de $L_2^{-1}L_1 = U_2U_1^{-1} = I$.

Pivoteo parcial

En el paso j , **antes** de eliminar, intercambiar la fila j con la fila $k \geq j$ que tenga el elemento de **módulo máximo** en la columna j :

$$|a_{kj}^{(j-1)}| = \max_{i \geq j} |a_{ij}^{(j-1)}|.$$

Así nunca se divide por cero y se divide por el pivote más grande disponible $\Rightarrow$ **menos propagación** de errores de redondeo.

El *pivoteo total* (máximo en toda la submatriz, permutando filas y columnas) casi no aporta beneficios frente al parcial, y es más costoso.

Código: LU con pivoteo parcial

```
function [L,U,p] = eleupiv(A)
% LU con pivoteo parcial:  $L*U = A(p,:)$ 
n = size(A,1);
L = zeros(n,n);
p = [1:n]';
for j = 1:n-1
    [M, k] = max(abs(A(j:n,j)));
    k = k+j-1; % fila del pivote
    p([k j]) = p([j k]); % registrar permutacion
    L([k j], :) = L([j k], :); % intercambiar filas
    A([k j], :) = A([j k], :);
    L(j+1:n,j) = A(j+1:n,j) / A(j,j);
    A(j+1:n,j) = 0;
    A(j+1:n,j+1:n) = A(j+1:n,j+1:n) - L(j+1:n,j)*A(j,j+1:n);
end
L = L + eye(n);
U = A;
```

Matriz de permutación elemental P : se intercambian dos filas de I . Si P intercambia filas j, k :

$$PP = I, \quad P^T = P,$$

$PA = A$ con filas j, k intercambiadas, $AP = A$ con columnas j, k intercambiadas.

Los pasos de Gauss con pivoteo:

$$U = A^{(n-1)} = G^{n-1}P^{n-1} \dots G^1P^1A.$$

Teorema ($PA = LU$)

Teorema

Con $U = A^{(n-1)}$ de Gauss con pivoteo,

$$PA = LU, \quad P = P^{n-1} \dots P^2 P^1,$$

donde U es tri. sup. y L es tri. inf. unitaria con $|\ell_{ij}| \leq 1$. La columna k de L (debajo de la diagonal) es una permutación de los multiplicadores del paso k .

Teorema ($PA = LU$)

Teorema

Con $U = A^{(n-1)}$ de Gauss con pivoteo,

$$PA = LU, \quad P = P^{n-1} \dots P^2 P^1,$$

donde U es tri. sup. y L es tri. inf. unitaria con $|\ell_{ij}| \leq 1$. La columna k de L (debajo de la diagonal) es una permutación de los multiplicadores del paso k .

Para resolver $Ax = b$: se permuta b ($\rightarrow Pb$) y se resuelven $Ly = Pb$, $Ux = y$.

`[L,U,p]=leupiv(A); y=triinf(L,b(p)); x=trisup(U,y)`

2.7 Matrices sdp: factorización de Cholesky

Matrices simétricas y definidas positivas (sdp)

$A \in \mathbb{R}^{n \times n}$ es **sdp** si

$$A^T = A \quad \text{y} \quad x^T A x > 0 \quad \forall x \in \mathbb{R}^n \setminus \{0\}.$$

(Semidefinida positiva si $x^T A x \geq 0$.)

Matrices simétricas y definidas positivas (sdp)

$A \in \mathbb{R}^{n \times n}$ es **sdp** si

$$A^T = A \quad \text{y} \quad x^T A x > 0 \quad \forall x \in \mathbb{R}^n \setminus \{0\}.$$

(Semidefinida positiva si $x^T A x \geq 0$.)

Propiedades que usaremos

- A sdp $\Rightarrow A$ es **invertible** ($Ax = 0 \Rightarrow x^T Ax = 0 \Rightarrow x = 0$).
- Autovalores reales y **positivos**; $a_{ii} = e_i^T A e_i > 0$.
- Si $A = \begin{bmatrix} \alpha & a^T \\ a & A_* \end{bmatrix}$, entonces $\alpha = a_{11} > 0$ y A_* es sdp.
- $R \in \mathbb{R}^{m \times n} \Rightarrow R^T R$ es simétrica semidef. positiva (sdp si las columnas de R son l.i.).

Teorema

Si $A \in \mathbb{R}^{n \times n}$ es sdp, existe R triangular superior tal que

$$A = R^T R.$$

*(De hecho: A es sdp **sii** existe R tri. sup. no singular con $A = R^T R$.)*

Teorema

Si $A \in \mathbb{R}^{n \times n}$ es *sdp*, existe R triangular superior tal que

$$A = R^T R.$$

(De hecho: A es *sdp* *sii* existe R tri. sup. no singular con $A = R^T R$.)

- R es el **factor de Cholesky**. Es un caso de LU con $L = R^T$, $U = R$ (con L no unitaria).
- Con diagonal **positiva**, R es **única**.
- Relación con LU : si $D = \text{diag}(R)$, entonces $A = \underbrace{R^T D^{-1}}_L \underbrace{DR}_U$.

Construcción (demostración constructiva)

Buscamos $R = \begin{bmatrix} \rho & r^T \\ 0 & R_* \end{bmatrix}$ con $A = \begin{bmatrix} \alpha & a^T \\ a & A_* \end{bmatrix} = \begin{bmatrix} \rho^2 & \rho r^T \\ \rho r & R_*^T R_* + rr^T \end{bmatrix}$.

Igualando bloques:

$$\rho = \sqrt{\alpha} > 0, \quad r = \frac{1}{\rho} a, \quad R_*^T R_* = A_* - rr^T.$$

Construcción (demostración constructiva)

Buscamos $R = \begin{bmatrix} \rho & r^T \\ 0 & R_* \end{bmatrix}$ con $A = \begin{bmatrix} \alpha & a^T \\ a & A_* \end{bmatrix} = \begin{bmatrix} \rho^2 & \rho r^T \\ \rho r & R_*^T R_* + rr^T \end{bmatrix}$.

Igualando bloques:

$$\rho = \sqrt{\alpha} > 0, \quad r = \frac{1}{\rho} a, \quad R_*^T R_* = A_* - rr^T.$$

El paso inductivo funciona porque $A_* - rr^T$ es **sdp**: tomando $x = \begin{bmatrix} -r^T y / \rho \\ y \end{bmatrix}$ se verifica

$$y^T (A_* - rr^T) y = x^T A x > 0.$$

Código: factorización de Cholesky

```
function R = cholesky(A)
% A sdp -> R triangular superior con A = R'*R
n = size(A,1);
R = zeros(size(A));
for i = 1:n-1
    R(i,i) = sqrt(A(i,i));           % fila i de R
    R(i,i+1:n) = A(i,i+1:n) / R(i,i);
    % actualizar el resto (hacia abajo)
    A(i+1:n,i+1:n) = A(i+1:n,i+1:n) - R(i,i+1:n)'*R(i,i+1:n);
end
R(n,n) = sqrt(A(n,n));
```

Actualizando solo la parte triangular superior, el costo se reduce a $\sim \frac{1}{3}n^3$ (mitad que LU). En

MATLAB/OCTAVE ya está: `R = chol(A)`.

- Triangular (L o U): resolver cuesta n^2 (sustitución adelante/atrás).
- Gauss = factorización $A = LU$: $\frac{2}{3}n^3$ operaciones.
- Con LU guardado, cada nuevo b cuesta solo $\sim 2n^2$.
- El pivoteo (parcial) garantiza $PA = LU$ y **controla** el error de redondeo.
- Si A es **sdp**: Cholesky $A = R^T R$, $\sim \frac{1}{3}n^3$ operaciones, **sin** pivoteo.

Para la próxima

Sensibilidad y **número de condición**; luego, métodos **iterativos** para sistemas grandes y ralos.